

Docker 101

Jordan Cleal

Docker 101

Docker packages your application and everything it needs to run - code, runtime, libraries, config - into a single portable unit called a container. Run that container anywhere Docker is installed and it behaves identically: the same on your laptop as in production, regardless of what else is installed on the host machine.

This solves the classic “works on my machine” problem. Your colleague running a different version of Python, your CI server running a different Linux distro, the production server with its own installed packages - none of that matters once your application is containerised.

What you will need: [Docker Desktop](#) installed on your machine. It is free to download for Mac, Windows, and Linux. After installation, verify it is working by opening a terminal and running `docker version`.

What this workshop covers:

- Part 1 - Concepts & building: containers vs VMs, images, layers, Dockerfile, `.dockerignore`, multi-stage builds
- Part 2 - Running & storage: `docker run` flags, container lifecycle, bind mounts, named volumes, the `node_modules` gotcha
- Part 3 - Debugging & registry: `docker logs`, `docker exec`, crashed containers, image naming, push/pull
- Part 4 - Scratch containers: `FROM scratch`, static linking, CA certificates, minimal images

Select a section from the sidebar to get started.

Containers vs VMs

	Virtual Machine	Container
Kernel	Full OS per VM	Shares host OS kernel
Isolation	Hypervisor emulates hardware	Kernel namespaces + cgroups
Boot time	Seconds to minutes	Milliseconds
Disk	GBs per image	MBs per image

Containers use Linux **namespaces** (PID, net, mnt, uts, ipc) to give each container its own view of the world, and **cgroups** to cap CPU and memory usage.

Images vs containers

- An **image** is a read-only blueprint - layers stacked on top of each other. Images are what you download from a registry or build yourself. Think of it as a recipe.
- A **container** is a running instance of an image. It adds a thin writable layer on top. Think of it as the meal made from the recipe.

```
# Pull an image (download layers)
```

```
docker pull nginx:alpine
```

```
# Run a container from that image
```

```
docker run --name web nginx:alpine
```

```
# List running containers
```

```
docker ps
```

```
# The image is unchanged - the container has its own writable layer
```

One image can run many containers simultaneously. Stopping a container does not delete its writable layer. Deleting a container discards that writable layer permanently.

Image layers

Every Dockerfile instruction that changes the filesystem creates a new layer. Metadata instructions (ENV, CMD, EXPOSE) create zero-byte layers.

```
FROM ubuntu:22.04          # base layer
RUN apt-get update \
  && apt-get install -y curl # new layer: packages added
COPY app/ /app              # new layer: your files
RUN chmod +x /app/run.sh    # new layer: permission change
CMD ["/app/run.sh"]         # metadata only (no layer)
```

Gotcha: RUN apt-get update on one line and RUN apt-get install on another creates a stale cache problem. Always combine them with &&.

Why layer order matters

```
# Slow - cache busted on any file change
FROM node:20-alpine
COPY . .          # copies everything
RUN npm install    # re-runs on ANY file change

# Fast - cache-friendly
FROM node:20-alpine
COPY package*.json ./ # only deps metadata
RUN npm install        # cached until pkg.json changes
COPY . .              # source changes don't bust npm cache
```

Rule: put things that change least often at the top. Dependencies before source code. Config before data.

```
# Inspect layers after building
docker history my-app:latest
docker image inspect my-app:latest | jq '.[0].RootFS.Layers'
```

Dockerfile core instructions

```
# Base image - always pin a specific tag in production
FROM node:20-alpine

# Working directory inside the container
WORKDIR /app

# Build-time variable (not in final image)
ARG NODE_ENV=production

# Runtime environment variable (baked into image)
ENV PORT=3000

# Copy files from host + image
COPY package*.json ./

# Run a command during build
RUN npm install --omit=dev

COPY . .

# Document the port (informational - doesn't publish it)
EXPOSE 3000
```

```
# Default executable - exec form, no shell wrapping
ENTRYPOINT ["node"]

# Default arguments - overridable at runtime
CMD ["server.js"]
```

CMD vs ENTRYPOINT

```
# Pattern 1: fixed command
CMD ["node", "server.js"]

# Pattern 2: entrypoint + overridable args
ENTRYPOINT ["node"]
CMD ["server.js"]          # override: docker run myimage other.js

# Pattern 3: shell script entrypoint (handles signals correctly)
COPY docker-entrypoint.sh /
ENTRYPOINT ["/docker-entrypoint.sh"]
CMD ["node", "server.js"]
```

Always use **exec form** ["executable", "arg"] - shell form `CMD node server.js` wraps in `/bin/sh -c`, which swallows `SIGTERM` and means your app never gets a graceful shutdown signal.

.dockerignore

Prevents files from being sent to the Docker build context. Faster builds, smaller images, no accidental secrets.

```
node_modules/      # huge - container builds its own
.git/              # version history not needed in image
.env               # secrets must never be baked in
.env.*
dist/              # build output (rebuilt inside container)
coverage/
*.log
.DS_Store
README.md
.github/
```

Security: a missing `.dockerignore` is the most common way secrets end up baked into images. Check `docker history` if you are unsure.

docker build

```
# Basic build
docker build -t my-app:latest .

# Build with a specific tag
docker build -t my-app:1.2.3 .

# Pass build args
docker build --build-arg NODE_ENV=staging -t my-app:staging .

# Build from a specific Dockerfile
docker build -f Dockerfile.dev -t my-app:dev .

# No cache (useful when deps change under you)
docker build --no-cache -t my-app:fresh .

# Multi-platform (for M1 Macs building for Linux amd64)
docker buildx build --platform linux/amd64 -t my-app:latest .
```

Multi-stage builds

Separate the build environment from the runtime environment. The builder stage can be huge - only the final stage ships.

```
FROM golang:1.22 AS builder

WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download

COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o server .

# Final stage - scratch has no shell, no libc, nothing
FROM scratch

COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /app/server /server

ENTRYPOINT ["/server"]
```

golang builder (~800MB) → scratch final image is just the binary (~5MB). No CVEs from the toolchain, no shell, minimal attack surface.

Try it - build a multi-stage image

```
# Create a minimal Go HTTP server
mkdir /tmp/scratch-demo && cd /tmp/scratch-demo
cat > main.go << 'EOF'
package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "hello from scratch")
    })
    log.Fatal(http.ListenAndServe(":8080", nil))
}
EOF
go mod init example.com/scratch-demo

# Build and check the size
docker build -t scratch-demo .
docker images scratch-demo

# Run it
docker run -d --name scratch-demo -p 8080:8080 scratch-demo
curl http://localhost:8080

# Clean up
docker rm -f scratch-demo
```

docker run - flags you'll use daily

```
docker run \  
  --name my-container \  
  -d \  
  -it \  
  -p 8080:80 \  
  -e DATABASE_URL=postgres://... \  
  -v $(pwd):/app \  
  -v my-data:/data \  
  --rm \  
  nginx:alpine
```

Container lifecycle

```
# Run → background  
docker run -d --name web nginx:alpine  
  
# See running containers  
docker ps  
  
# See all containers (including stopped)  
docker ps -a  
  
# Stop (sends SIGTERM, waits 10s, then SIGKILL)  
docker stop web  
  
# Start again  
docker start web  
  
# Remove (must be stopped first)  
docker rm web  
  
# Force-remove (stop + rm in one)  
docker rm -f web  
  
# Remove all stopped containers  
docker container prune
```

Try it - nginx on port 8080

```
# Run nginx in the background, map port 8080 → 80  
docker run -d --name my-nginx -p 8080:80 nginx:alpine  
  
# Verify it's running  
docker ps  
  
# Hit it  
curl http://localhost:8080  
# or open http://localhost:8080 in a browser  
  
# Check logs  
docker logs my-nginx  
docker logs -f my-nginx # follow (like tail -f)  
  
# Clean up  
docker stop my-nginx  
docker rm my-nginx
```

COPY vs volumes

COPY bakes files into the image at build time. Fast. Immutable. Requires a rebuild to update.

Volume / bind mount mounts external storage at runtime. Live updates. Not part of the image.

COPY: file is baked in - editing the host file does nothing
docker run my-app *# uses /app/config.json from the image*

Bind mount: host file is live-mounted
docker run -v \$(pwd)/config.json:/app/config.json my-app
edit config.json on the host → container sees it immediately

Named volume: Docker manages the storage location
docker run -v my-db-data:/var/lib/postgresql/data postgres
data persists across container restarts and removes

The node_modules gotcha

This looks right - live-reload source code during dev
docker run -v \$(pwd):/app node-app

But it MASKS /app/node_modules from the container!
The host probably has: node_modules/ (wrong arch, or missing)
The container had: /app/node_modules (built correctly with npm install)
Mounting \$(pwd) over /app hides the container's node_modules

The bind mount replaces the entire /app directory. Anything that was in the container at /app - including node_modules - is hidden.

The fix - anonymous volume trick

```
docker run \  
  -v $(pwd):/app \           # bind mount source code  
  -v /app/node_modules \     # anonymous volume: preserves container's node_modules  
  -p 3000:3000 \  
  node-app
```

Docker resolves mount precedence: the anonymous volume at /app/node_modules takes priority over the bind mount above. The container sees its own correctly-built node_modules, plus your live source files.

In docker-compose.yml:

```
services:  
  app:  
    build: .  
    ports:  
      - "3000:3000"  
    environment:  
      - NODE_ENV=development  
    volumes:  
      - ./app           # live code reload  
      - /app/node_modules # preserve container's node_modules
```

Named volumes for persistence

Named volume - data survives container removal
docker run -d \
 --name postgres \
 -e POSTGRES_PASSWORD=secret \
 -v pg-data:/var/lib/postgresql/data \
 postgres:16-alpine

```

# Remove and recreate - data is still there
docker rm -f postgres
docker run -d --name postgres \
  -e POSTGRES_PASSWORD=secret \
  -v pg-data:/var/lib/postgresql/data \
  postgres:16-alpine

# Manage volumes
docker volume ls
docker volume inspect pg-data
docker volume rm pg-data      # explicitly delete when done

```

Try it - bind mounts & the node_modules fix

```

# 1. Run nginx serving a local HTML file via bind mount
mkdir -p /tmp/html
echo "<h1>hello from the host</h1>" > /tmp/html/index.html
docker run -d --name vol-demo -p 8080:80 \
  -v /tmp/html:/usr/share/nginx/html:ro \
  nginx:alpine
curl http://localhost:8080
# Edit /tmp/html/index.html and curl again - live update, no rebuild

# 2. See the node_modules gotcha in action
docker run --rm -v $(pwd):/app node:20-alpine sh -c \
  "ls /app/node_modules 2>/dev/null || echo 'node_modules is GONE'"

# 3. Fix it with the anonymous volume
docker run --rm -v $(pwd):/app -v /app/node_modules node:20-alpine sh -c \
  "ls /app/node_modules 2>/dev/null && echo 'node_modules is intact' || echo 'still gone'"

# Clean up
docker rm -f vol-demo

```

docker logs

```

# Print all logs
docker logs my-container

# Follow (tail -f equivalent)
docker logs -f my-container

# Last N lines
docker logs --tail 50 my-container

# With timestamps
docker logs -t my-container

# Since a point in time
docker logs --since 5m my-container
docker logs --since 2024-01-15T10:00:00 my-container

# Combine: follow last 20 lines with timestamps
docker logs -f -t --tail 20 my-container

```

Logs come from container stdout/stderr. If your app writes to a file instead, you will not see anything here. Always write to stdout in containers.

docker exec

```
# Open a shell in a running container
docker exec -it my-container sh      # Alpine / minimal images
docker exec -it my-container bash    # Debian/Ubuntu-based

# Run a one-off command
docker exec my-container ls /app
docker exec my-container cat /etc/os-release

# Common debugging patterns
docker exec -it my-container sh -c "env | grep DATABASE"
docker exec -it my-container sh -c "ps aux"
docker exec -it my-container sh -c "ls -la /app"
docker exec -it my-container sh -c "curl -s localhost:3000/health"

# Check what process is PID 1 (the one that gets SIGTERM)
docker exec my-container cat /proc/1/cmdline
```

Debugging a crashed container

```
# Container exited - check the exit code
docker ps -a
# EXITED (1) means the process failed
# EXITED (137) means OOM killed (or SIGKILL)

# Check logs of a stopped container
docker logs my-container

# Inspect the container metadata
docker inspect my-container | jq '.[0].State'

# Run the image with a shell override (skip CMD/ENTRYPOINT)
docker run -it --entrypoint sh my-app
```

Scratch containers have no shell - `docker exec` will not work. Build a separate `:debug` image tag that uses `FROM busybox` as the final stage instead of `FROM scratch`.

Try it - debug a running container

```
# Start a container
docker run -d --name debug-me nginx:alpine

# Explore from inside
docker exec -it debug-me sh

# Inside the container:
# ps aux           - what's running
# env              - environment variables
# ls /etc/nginx     - nginx config location
# cat /etc/nginx/nginx.conf
# exit

# Check resource usage
docker stats debug-me  # live CPU/memory

# Clean up
docker rm -f debug-me
```

Image naming

registry/repository:tag

docker.io/library/nginx:alpine

```
        tag (version/variant)
      image name
    user/org (library = Docker official)
  registry host (docker.io = Docker Hub)
```

Short form (defaults apply)

```
nginx          → docker.io/library/nginx:latest
nginx:alpine   → docker.io/library/nginx:alpine
myuser/myapp   → docker.io/myuser/myapp:latest
```

Tag, push, pull

Build and tag

```
docker build -t myapp:1.0.0 .
```

Tag with a registry prefix

```
docker tag myapp:1.0.0 myregistry.example.com/myteam/myapp:1.0.0
```

Push to registry (must be logged in)

```
docker push myregistry.example.com/myteam/myapp:1.0.0
```

Pull on another machine

```
docker pull myregistry.example.com/myteam/myapp:1.0.0
```

Login to a registry

```
docker login myregistry.example.com
```

or with credentials

```
echo $TOKEN | docker login myregistry.example.com -u $USER --password-stdin
```

Try it - tag and inspect

Pull an image and inspect its full name

```
docker pull nginx:alpine
```

```
docker inspect nginx:alpine | jq '[0].RepoDigests'
```

Build a tiny image and tag it

```
cat > /tmp/Dockerfile.hello << 'EOF'
```

```
FROM alpine:latest
```

```
CMD ["echo", "hello from my-registry"]
```

```
EOF
```

```
docker build -f /tmp/Dockerfile.hello \
  -t my-registry.example.com/demo/hello:1.0.0 .
```

Verify the tag appears

```
docker images | grep my-registry
```

Add a second tag (same image, different name)

```
docker tag my-registry.example.com/demo/hello:1.0.0 \
  my-registry.example.com/demo/hello:latest
```

```
docker images | grep hello
```

Inspect before pushing (dry-run)

```
docker image inspect my-registry.example.com/demo/hello:1.0.0 \
  | jq '[0] | {Id, RepoTags, Size}'
```

```
# Clean up
docker rmi my-registry.example.com/demo/hello:1.0.0 \
    my-registry.example.com/demo/hello:latest
```

What is FROM scratch?

FROM scratch is a special Docker keyword - it means “start with an empty filesystem.” No shell, no libc, no package manager, no utilities.

```
FROM scratch
COPY my-binary /app
ENTRYPOINT ["/server"]
```

- The resulting image contains *only* what you explicitly COPY in
 - No shell → `docker exec -it ... sh` fails (nothing to exec)
 - No libc → dynamically linked binaries will segfault at startup
 - Minimal CVE surface → security scanners have almost nothing to report
-

Static vs dynamic linking

Most languages (C, Python, Ruby) link against glibc at runtime. If glibc is not in the image, the binary crashes immediately.

```
# Dynamic binary - doesn't work on scratch
ldd ./my-app
# libpthread.so.0 => /lib/...
# libc.so.6 => /lib/...

# Go with CGO disabled - statically linked, works on scratch
CGO_ENABLED=0 go build -o server .
ldd ./server
# not a dynamic executable
```

Other languages that work on scratch: Rust (statically links by default), C compiled with musl.

The CA certificates gotcha

Your scratch binary makes an HTTPS call and gets:

```
x509: certificate signed by unknown authority
```

Why: `/etc/ssl/certs/ca-certificates.crt` does not exist on a scratch image - there is nothing there.

Fix: copy the cert bundle from the builder stage before it is thrown away.

```
FROM golang:1.22 AS builder
RUN CGO_ENABLED=0 go build -o server .

FROM scratch
# Without this line, any HTTPS request fails
COPY --from=builder /etc/ssl/certs/ca-certificates.crt \
    /etc/ssl/certs/
COPY --from=builder /app/server /server
ENTRYPOINT ["/server"]
```

Other files you may need on scratch

```
FROM scratch

# TLS certificate bundle (HTTPS calls)
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# Timezone data (if your app formats local times)
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo
```

```
# /etc/passwd (if your app looks up the current user)
COPY --from=builder /etc/passwd /etc/passwd
COPY --from=builder /etc/group /etc/group

# Your binary
COPY --from=builder /app/server /server
ENTRYPOINT ["/server"]
```

Full scratch Dockerfile

```
FROM golang:1.22 AS builder

WORKDIR /app
COPY . .

# CGO_ENABLED=0 = statically linked binary
# -ldflags="-s -w" strips debug symbols (smaller binary)
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o server .

FROM scratch

COPY --from=builder /etc/ssl/certs/ca-certificates.crt \
    /etc/ssl/certs/
COPY --from=builder /app/server /server

ENTRYPOINT ["/server"]
```

~800MB build image → ~6MB scratch image. The attack surface shrinks from hundreds of packages to zero.

Try it - build a scratch image

```
mkdir /tmp/scratch-demo && cd /tmp/scratch-demo

cat > main.go << 'EOF'
package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "hello from scratch")
    })
    log.Fatal(http.ListenAndServe(":8080", nil))
}
EOF
go mod init example.com/scratch-demo

# Build and check the size
docker build -t scratch-demo .
docker images scratch-demo

# Run it
docker run -d --name scratch-demo -p 8080:8080 scratch-demo
curl http://localhost:8080

# Try to exec in (will fail - no shell)
```

```
docker exec -it scratch-demo sh    # OCI runtime exec failed
```

```
# Clean up
```

```
docker rm -f scratch-demo
```

Key gotchas to remember

Problem	Cause	Fix
<code>node_modules</code> missing in dev container	Bind mount masks the container's <code>/app</code> directory	Add <code>-v /app/node_modules</code> anonymous volume
Binary crashes on scratch	Dynamically linked against glibc	Set <code>CGO_ENABLED=0</code> or use musl - check with <code>ldd</code>
HTTPS fails on scratch	No CA cert bundle	Copy <code>ca-certificates.crt</code> from the builder stage
SIGTERM not received	CMD in shell form	Use exec form <code>["node", "server.js"]</code>

Where to go next

- [Docker Compose](#) - multi-container apps locally
- [Kubernetes](#) - container orchestration at scale
- [AWS ECS](#) - managed containers on AWS
- [Trivy](#) - vulnerability scanner for images
- [dive](#) - interactive layer inspector